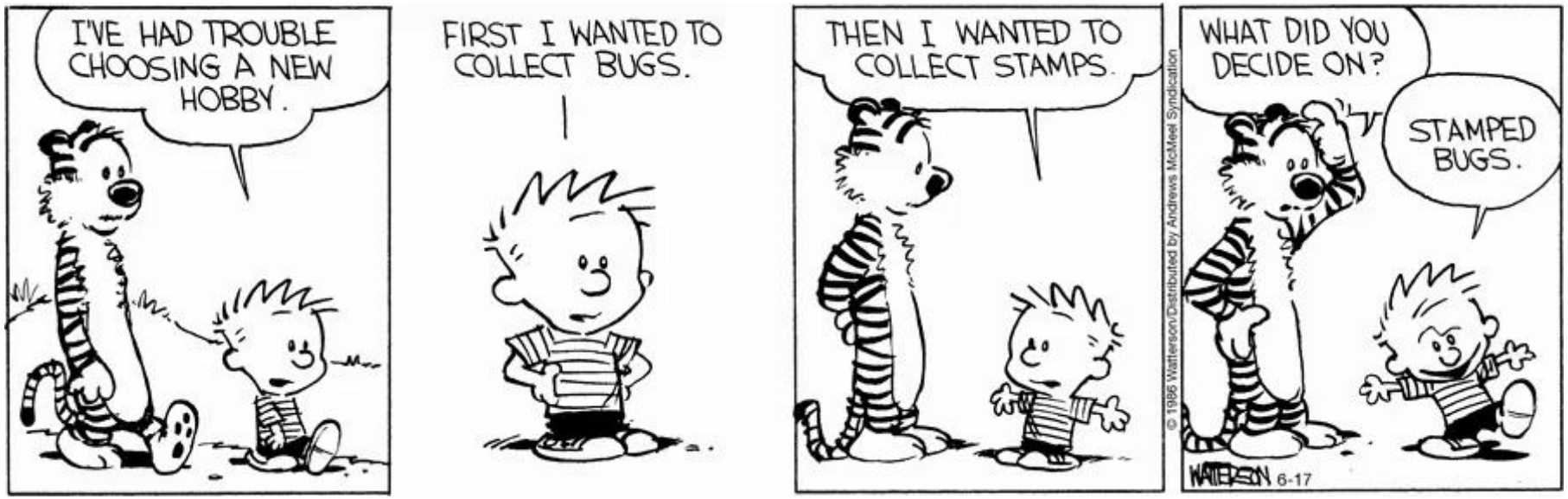
Lecture 16:
Reading + Writing Files



INFO 1100: Introduction to Programming

we've learned a lot of ways to use Python for analyzing data

we've learned a lot of ways to use Python for analyzing data

but we're missing ***a critical element****!*

we've learned a lot of ways to use Python for analyzing data

but we're missing ***a critical element****!*

loading in data

we've learned a lot of ways to use Python for analyzing data

*but we're missing **a critical element**!*

loading in data

saving outputs

we've learned a lot of ways to use Python for analyzing data

*but we're missing **a critical element**!*

~~loading in data~~

reading files

saving outputs

we've learned a lot of ways to use Python for analyzing data

*but we're missing **a critical element**!*

~~loading in data~~

reading files

~~saving outputs~~

writing files

Opening files

Opening files

```
open(<file-path>)
```

Opening files

`open(<file-path>)`

returns a file object

Opening files

`open(<file-path>)`

returns a file object

not the file itself

Opening files

you can also indicate the **mode**
in which the file is opened

Opening files

you can also indicate the **mode**
in which the file is opened

```
open(<file-path>, mode=<mode>)
```

Opening files

you can also indicate the **mode**
in which the file is opened

```
open(<file-path>, mode=<mode>)
```

what operations you can do with the file object

Opening files

you can also indicate the **mode**
in which the file is opened

`r`	reading text
`w`	writing texts – *clears file contents*
`a`	writes to end of file – *appends*
`x`	writing texts – *if they don't exist*
`+`	reading *and* writing texts

Opening files

you can also indicate the **mode**
in which the file is opened

you can add **`b`** *to any mode*
to read as binary data

Opening files

you can also indicate the **mode**
in which the file is opened

```
f = open("file.txt", mode="r")
```

to read

Opening files

you can also indicate the **mode**
in which the file is opened

```
f = open("file.txt", mode="w")
```

to write

Opening files

you can also indicate the **mode**
in which the file is opened

```
f = open("file.txt", mode="w")
```

to write

remember: this overwrites file.txt

Opening files

you can also indicate the **mode**
in which the file is opened

```
f = open("file.txt", mode="w")
```

to write

you can append or write to a new file instead

Opening files

you can also indicate the **mode**
in which the file is opened

```
f = open("file.txt", mode="w")
```

to write

you *should* append or write to a new file instead

Closing files

Closing files

we also need to **close** file objects
we've opened

Closing files

we also need to **close** file objects
we've opened

content might not be
written till file closed

Closing files

we also need to **close** file objects
we've opened

content might not be
written till file closed

open files use up
memory

Closing files

```
f = open("file.txt", mode="w")
```

Closing files

```
f = open("file.txt", mode="w")
```

```
f.close()
```

Closing files

```
f = open("file.txt", mode="w")
```

```
f.close()
```

easy as that!

Closing files

if we want to avoid having to remember to close our files…

Closing files

if we want to avoid having to remember to close our files...

```
f = open("file.txt", mode="w")
```

Closing files

if we want to avoid having to remember to close our files...

```
f = open("file.txt", mode="w")
```

```
with open("file.txt", mode="w") as f:
    <code>
```

Closing files

if we want to avoid having to remember to close our files...

```
with open("file.txt", mode="w") as f:
    <code>
```

automatically closes after **<code>** *runs*

Reading files

Reading files

we've opened a file object...

Reading files

we've opened a file object...

*but how do we **read** that object?*

Reading files

several file object methods

Reading files

several file object methods

`read`	read whole file as string or bytes
`readline`	read the next line of file
`readlines`	read all lines of text from the file

Reading files

fruit.txt

```
banana
apple
kiwi
pineapple
```

Reading files

fruit.txt

```
banana
apple
kiwi
pineapple
```

Reading files

fruit.txt

```
banana
apple
kiwi
pineapple
```

```
with open("fruit.txt", "r") as f:
    <code>
```

Reading files

fruit.txt

```
banana
apple
kiwi
pineapple
```

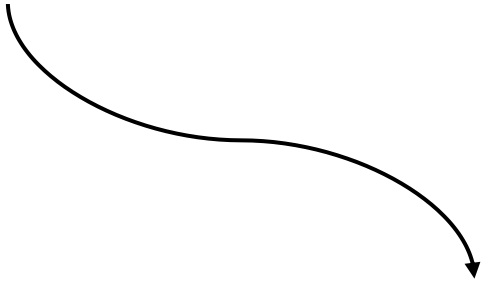
```
with open("fruit.txt", "r") as f:
    text = f.read()
```

Reading files

fruit.txt

```
banana
apple
kiwi
pineapple
```

```
with open("fruit.txt", "r") as f:
    text = f.read()
```



```
"banana\napple\nkiwi\npineapple"
```

Reading files

fruit.txt

```
banana
apple
kiwi
pineapple
```

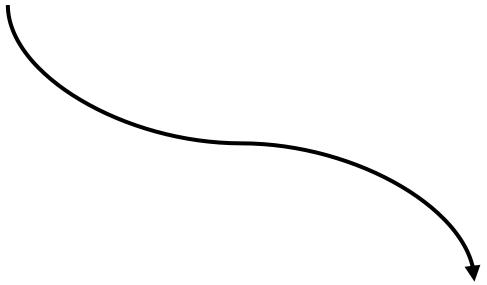
```
with open("fruit.txt", "r") as f:
    text = f.readline()
```

Reading files

fruit.txt

```
banana
apple
kiwi
pineapple
```

```
with open("fruit.txt", "r") as f:
    text = f.readline()
```



```
"banana\n"
```

Reading files

fruit.txt

```
banana
apple
kiwi
pineapple
```

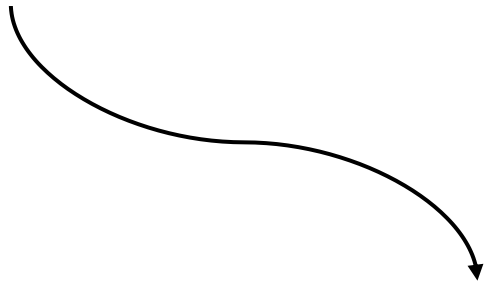
```
with open("fruit.txt", "r") as f:
    text = f.readlines()
```

Reading files

fruit.txt

```
banana
apple
kiwi
pineapple
```

```
with open("fruit.txt", "r") as f:
    text = f.readlines()
```



```
["banana\n", "apple\n", "kiwi\n",
 "pineapple\n"]
```

Reading files

you can also for-loop through file lines

Reading files

fruit.txt

```
banana
apple
kiwi
pineapple
```

Reading files

fruit.txt

```
banana
apple
kiwi
pineapple
```

```
with open("fruit.txt", "r") as f:
    <code>
```

Reading files

fruit.txt

```
banana
apple
kiwi
pineapple
```

```
with open("fruit.txt", "r") as f:
    for line in f:
        print(line)
```

Reading files

fruit.txt

```
banana
apple
kiwi
pineapple
```

```
with open("fruit.txt", "r") as f:
    for line in f:
        print(line)
```

still has newlines!

Reading files

fruit.txt

```
banana
apple
kiwi
pineapple
```

```
with open("fruit.txt", "r") as f:
    for line in f:
        print(line.strip())
```

Reading files

fruit.txt

```
banana
apple
kiwi
pineapple
```

```
with open("fruit.txt", "r") as f:
    for line in f:
        print(line.strip())
```

you can only loop through a file object **once**!

Writing files

Writing files

now, let's try *writing* to files!

Writing files

several file object methods

Writing files

several file object methods

`write`	write a string to the file
`writelines`	write an iterable of strings to the file

Writing files

```
with open("fruit.txt", "w") as f:
    <code>
```

Writing files

```
with open("fruit.txt", "w") as f:
    f.write(<str>)
```

Writing files

```
with open("fruit.txt", "w") as f:
    f.write("banana\napple\nkiwi
        \npineapple\n")
```

Writing files

```
with open("fruit.txt", "w") as f:
    f.write("banana\napple\nkiwi
    \npineapple\n")
```

remember that you need to include newlines!

Writing files

```
with open("fruit.txt", "w") as f:
    f.writelines(<iterable>)
```

Writing files

```
with open("fruit.txt", "w") as f:
    f.writelines(["banana",
                  "apple", "kiwi",
                  "pineapple"])
```

Useful module

Useful module

the `os` module has a lot of functionality
useful for working with files

Useful module

let's look at some useful functions!

Useful module

`os.getcwd()`

finds the current working directory

Useful module

`os.listdir(<dir>)`

lists everything in the directory `<dir>`

Useful module

`os.exists(<path>)`

checks whether the file or directory at `<path>` at exists

Useful module

`os.join(<path1>, <path2>, …)`

combines all `<pathn>` strings into a path for your system

Example 1

```
total = 0
count = 0
with open("numbers.txt") as f:
    for line in f:
        n = int(line.strip())
        if n > 8:
            total += n
            count += 1
print(total)
print(count)
```

numbers.txt

```
10
25
7
```

Example 2

```
counts = {}
with open("authors.txt") as f:
    for line in f:
        name = line.strip()
        if name in counts:
            counts[name] += 1
        else:
            counts[name] = 1
print(counts)
print(len(counts))
```

authors.txt

```
Austen
Christie
Austen
Dumas
```

Example 3

```
with open("notes.txt") as f:
    text = f.read()

with open("notes.txt", "w") as f:
    f.write("done")

with open("notes.txt") as f:
    print(f.read())

print(len(text))
```

notes.txt

```
hello
world
```

Example 4

```
def bump(source, dest):
    with open(source) as f:
        lines = f.readlines()

    with open(dest, "w") as f:
        for line in lines:
            f.write(str(int(line.strip()) + 1) + '\n')

bump("scores.txt", "new_scores.txt")
with open("new_scores.txt") as f:
    print(f.read())
```

scores.txt

```
88
92
79
```

Questions?

(… I'll wait for 2)

For tomorrow:

(07/24 @ 11:30am)

- Lab 16
- Quiz tomorrow!

For tomorrow:

(07/24 @ 11:30am)

- Lab 16
- Quiz tomorrow!

Monday:

(07/27 @ 9:00am)

Final Project Proposal

Tuesday:

(07/28 @ 11:30am)

Readings + GRQs